

THERMAL-AWARE PATH PLANNER

Distance- and Temperature-Aware Robot Navigation in Industrial Environments

Abdulmlek Al-Sharee | Mechatronics Engineering | Sana'a University

Object-Oriented Programming with C++ — Final Project

PROJECT IDEA

Don't just find A path. Find a SAFE one.

A robot moving through an industrial space shouldn't only ask “what's the shortest route?” It should also ask “what will this route expose me to?”

A Path A — Shortest

Minimizes distance only. Cuts straight past a heat source — exposing electronics, battery, and sensors to unnecessary thermal stress.

✗ **Fast, but risky near equipment**

B Path B — Thermal-Aware

Slightly longer, but weighs distance AND thermal exposure together — detouring around hazardous zones automatically.

✓ **This is what this project builds**

PROBLEM STATEMENT

Traditional planners are thermally blind

1

Free space vs. obstacles

Classic path planning only asks: is this cell blocked or not.

2

Heat is invisible to it

A furnace, motor, or hot duct isn't a physical obstacle — so it's simply never considered.

3

Real cost to the robot

Repeated exposure degrades batteries, electronics, and sensors over time.

How can a planner keep peak thermal exposure low while still reaching the goal efficiently — without redesigning the search algorithm for every new hazard?

OBJECTIVES

What this project set out to do

1. Separate spatial data, graph structure, and search logic into independent OOP modules
2. Make the cost function pluggable — new planning criteria without touching A* itself
3. Correctly ingest a real thermal dataset exported from a SolidWorks thermal study
4. Build a navigable graph from irregular, simulation-derived data — not a perfect grid
5. Prove, quantitatively, that thermal-aware planning beats distance-only planning
6. Export results for downstream analysis and visualization

MAIN FEATURES

What the system actually does

3D Geometry

Point3D with full arithmetic & comparison operators

Irregular Graph Building

Auto-connects a real, unevenly-spaced mesh

Swappable Cost Functions

DistanceCost ↔ ThermalCost via polymorphism

Thermal-Aware A*

Full open/closed-list search with live cost tracking

Real SolidWorks Ingestion

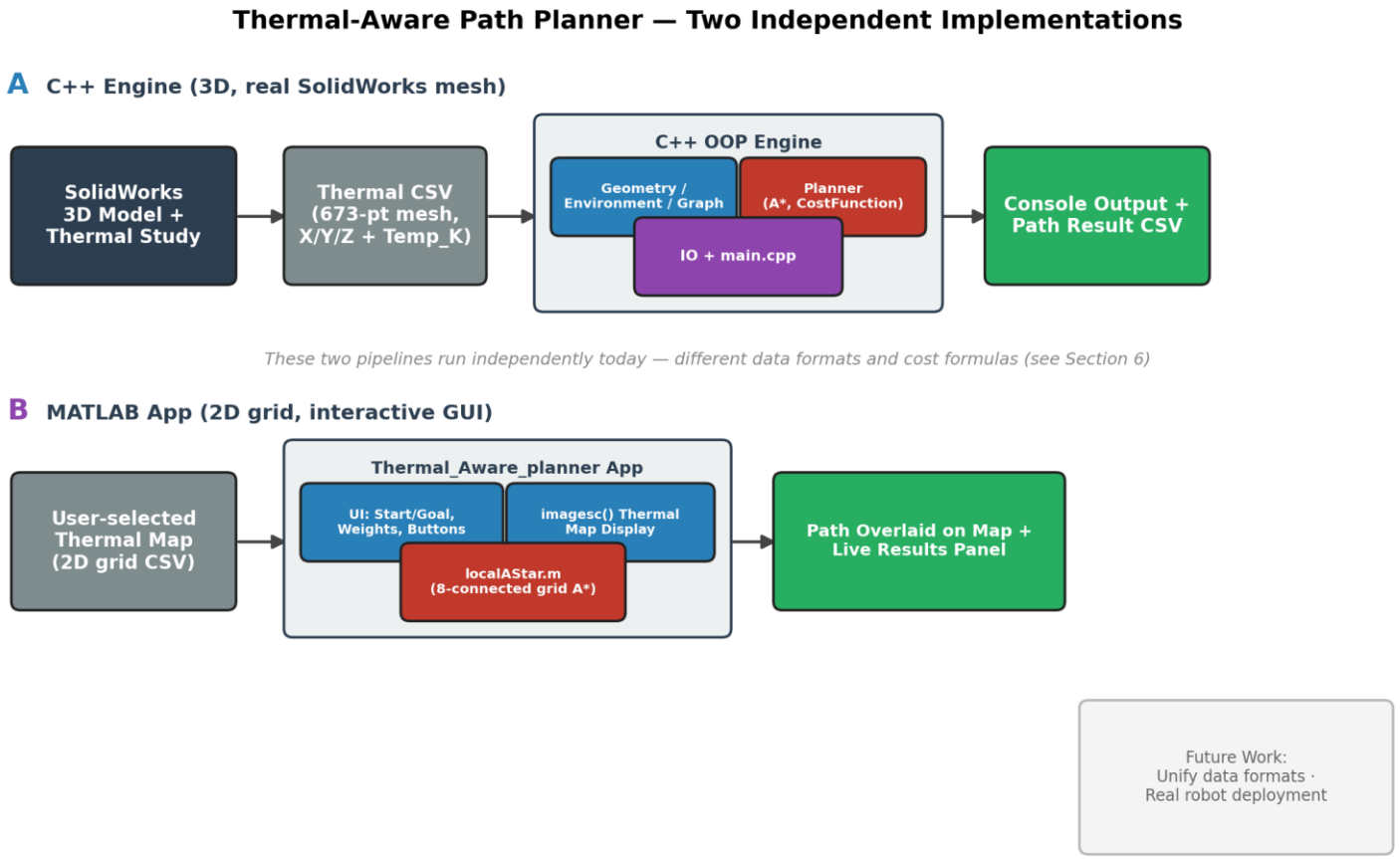
Parses actual thermal-study CSV exports

Interactive MATLAB App

Load a map, set weights, visualize the path live

SYSTEM DESIGN

Two implementations, one idea



C++ CONCEPTS USED

Real OOP, applied deliberately

Encapsulation

Private fields, public getters/setters

Inheritance & Polymorphism

CostFunction → Distance/ThermalCost, runtime dispatch

Abstract Base Classes

Pure virtual calculate() + virtual destructor

RAII / Smart Pointers

unique_ptr<CostFunction>, no manual delete

Exception Handling

try/catch around every CSV row

Operator Overloading

Point3D: ==, !=, +, -, <<

STL Containers & Algorithms

vector, min_element, find, reverse

File I/O

ifstream / ofstream for all CSV read & write

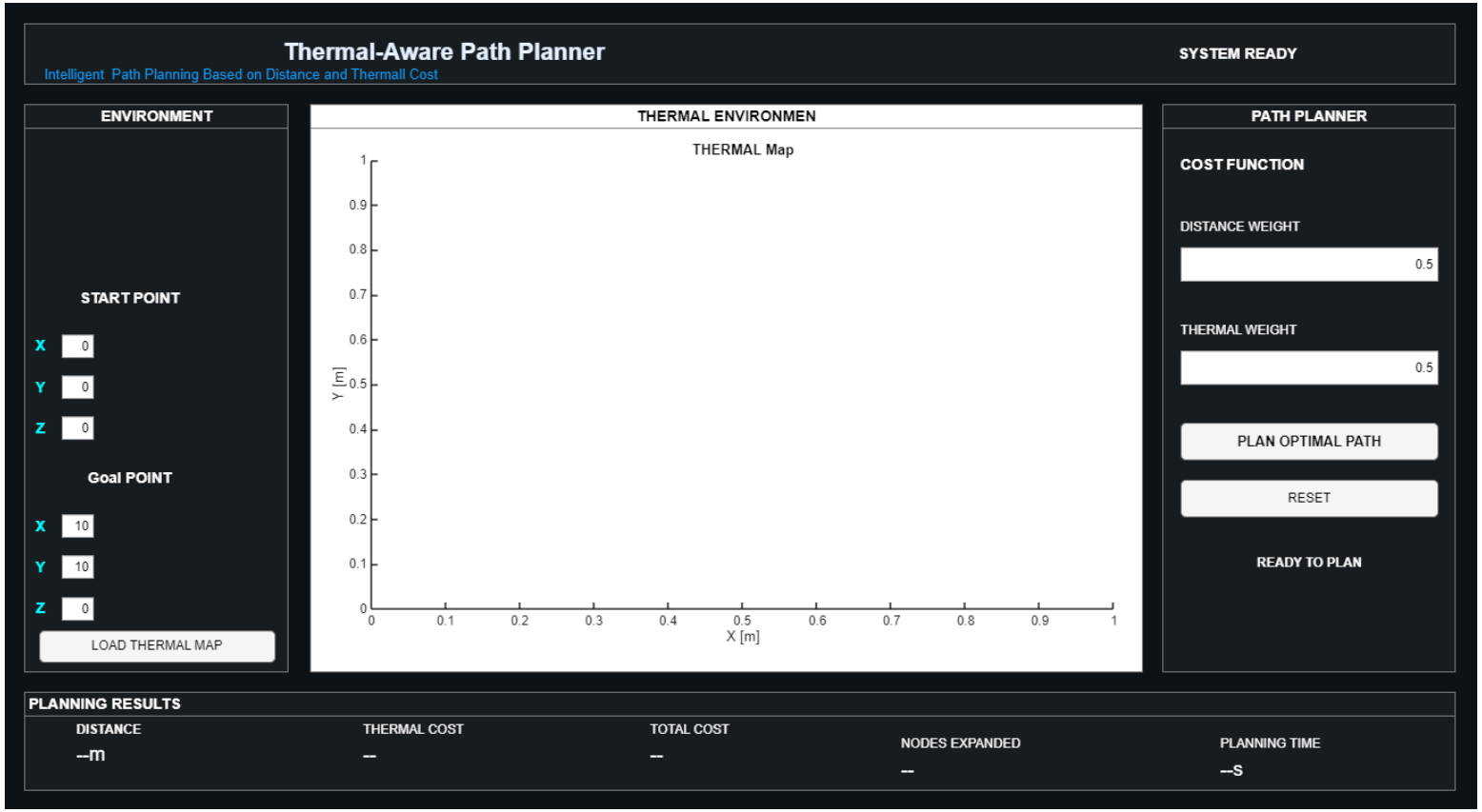
IMPLEMENTATION HIGHLIGHTS

The parts doing the real work

Graph::getNeighbors()	Finds each node's 6 nearest real neighbors by distance — no assumption of a perfect grid.
CostFunction hierarchy	One virtual calculate() call; swapping strategy = swapping one line in main.cpp.
AStar::findPath()	Standard open/closed-list search; reconstructPath() walks parent pointers back to start.
ThermalDataReader	Parses the real SolidWorks export, converts Kelvin → Celsius, guards bad rows with try/catch.

INTERACTIVE FRONT-END

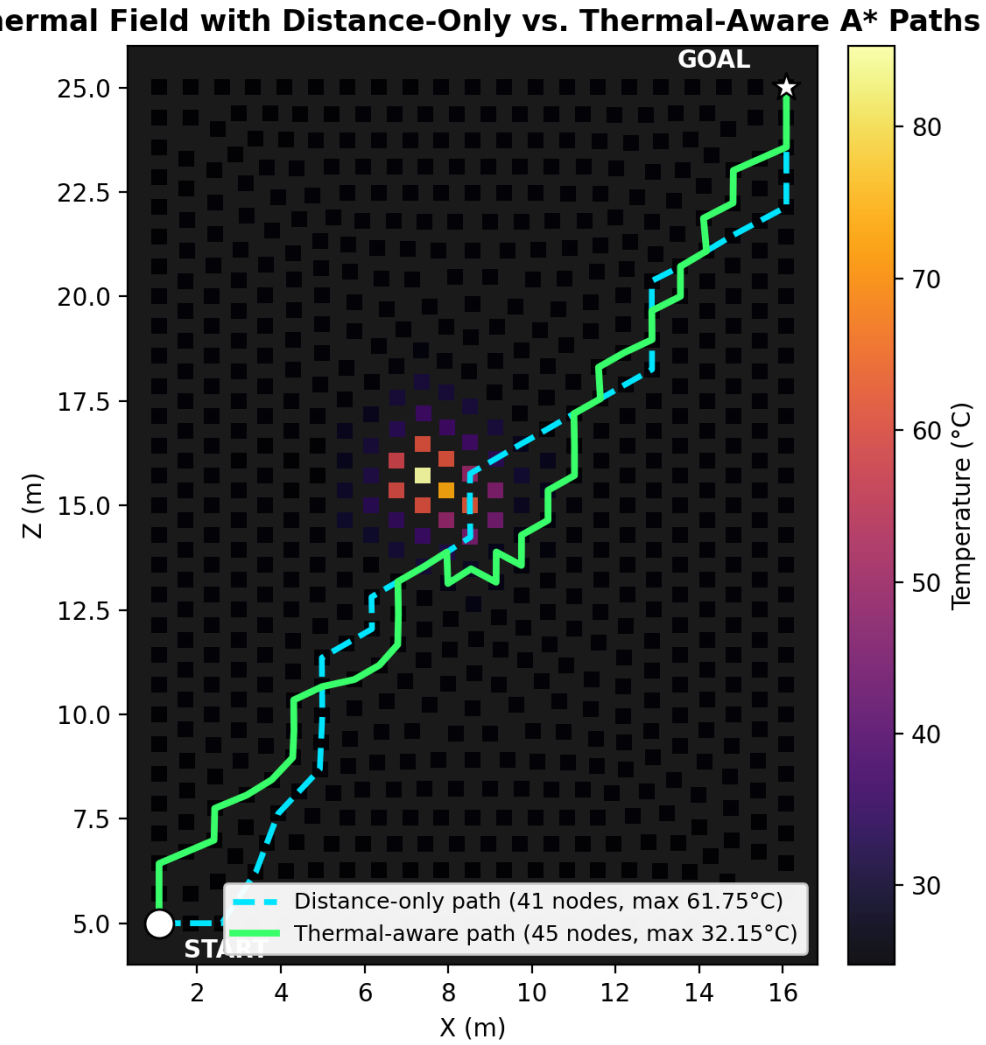
MATLAB App: plan and visualize live



- Load any 2D thermal map (CSV grid)
- Set Start / Goal points
- Tune Distance vs. Thermal weight
- localAStar.m computes the path
- Live distance, cost & timing readout

RESULTS

It actually avoids the heat



RESULTS SUMMARY

Same start, same goal, same real data

41

NODES

Distance-only path

61.75°C peak

45

NODES

Thermal-aware path

32.15°C peak

~10% longer path → 47% lower peak thermal exposure

CHALLENGES FACED

Three bugs stood between “compiles” and “works”

1

CSV Column Mismatch

Reader assumed the wrong column order from the real SolidWorks export — every position & temperature was wrong.

2

Neighbor Distance Too Small

0.01-unit tolerance vs. a real ~0.714-unit mesh spacing — every node was isolated.

3

Missing Output Folder

ofstream doesn't create missing directories — export silently failed on a clean checkout.

Every fix was verified empirically — diagnostic scripts, measured real spacing, and a clean-checkout re-run — not just code review.

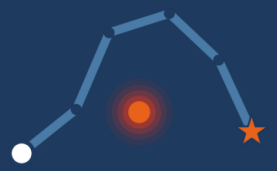
CONCLUSION

A small change in cost, a large change in safety

Swapping one polymorphic cost function — without touching the A* algorithm itself — turns a purely geometric planner into one that actively avoids real thermal hazards, verified on real SolidWorks simulation data.

NEXT STEPS

- Unify the C++ and MATLAB implementations on one data format
- Swap the linear open-list scan for a std::priority_queue
- Deploy on a real robot: Raspberry Pi, motor drivers, live temperature sensing



Thank You

Questions?